

# SOFTWARE REQUIREMENT SPECIFICATION (SRS)

## PROJECT: SKAVS CORE ENGINE (SYSTEM-WIDE KNOWLEDGE & ACADEMIC VIRTUAL SPECIALIST)

Document Number: **HOYE-SKAVS-2026-V1**    Status: **ARCHITECTURAL DESIGN PHASE**

Institution: **Hoye Secondary School**

Target Environment: **VS Code / Node.js Production Server**

Date: **May 30, 2026**

Authoring Architecture: **Enterprise System Architecture**

## 1. EXECUTIVE SUMMARY & IDENTITY DEFINITION

### 1.1 Identity Profile

**SKAVS** is the **System-Wide Knowledge & Academic Virtual Specialist**. It is a lightweight, asynchronous conversational Artificial Intelligence application custom-engineered specifically for the educational ecosystem of Hoye Secondary School.

Unlike legacy educational portals that act as passive data grids or demand complex administrative data logging, SKAVS operates as a live, intelligent operational concierge. The machine dynamically modulates its systemic behavior, strict logical constraints, and conversational tone based upon the explicit user authentication classification selected at the initialization gate.

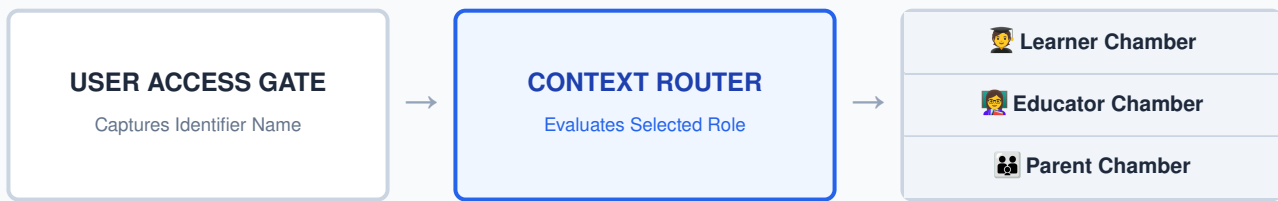
### 1.2 Strategic Core Mission

To directly eliminate resource discrepancies, overcrowding disadvantages, and administrative exhaustion within a developing or rural school structure. SKAVS achieves this by supplying instant 24/7 localized administrative discovery for parents, comprehensive lesson engineering frameworks for educators, and individualized Socratic coaching protocols for learners. It accomplishes this via an optimized Single Page Application pipeline designed to perform efficiently even over low-bandwidth cellular data arrays.

## 2. FUNCTIONAL MATRIX & CONTEXT CHAMBERS

Before granting database execution or core model access, the application triggers a primary context gate. This layer prompts the user for their name to maintain personalized state strings throughout the processing lifecycle, followed by a segmented classification selector.

FIGURE 2.1: CONTEXT GATE & SESSION ROUTING INFRASTRUCTURE



## 2.1 The Learner Chamber (Personal AI Tutor)

- **Objective:** Deliver hyper-personalized, continuous tutoring to students without engineering a systemic dependency on automated answer-copying frameworks.
- **Strict Structural Constraint:** The prompt validation engine completely blocks the rendering of direct homework answers. If standard syllabus lookups are requested, the engine enforces the **Socratic Method**.
- **Execution Framework:** The agent isolates the target curriculum objective, provides contextual analogies calibrated for local student settings (e.g., matching algebraic logic to school sports parameters or layouts), and leads the student toward independently calculating the correct answers through iterative hint intervals.

## 2.2 The Educator Chamber (The Lesson Architect)

- **Objective:** Mitigate teacher exhaustion and provide granular class performance insights back to human instructors.
- **Syllabus & Material Deconstruction:** Teachers insert dense curriculum source sheets or slide outlines. SKAVS returns a standardized 45-minute lesson blueprint (e.g., 10-minute engagement hook, 20-minute key mechanics presentation, 15-minute formative review loop).
- **Realistic Pacing Matrix:** Educators define clear constraints (e.g., 6 aggregate teaching hours across 4 business days). The algorithm calculates and balances a practical schedule adhering to standard public school realities.
- **Learner FAQ Aggregator:** The backend dynamically aggregates and groups unrecognized or trending topics tracked in the Learner Chamber. When teachers enter their module, SKAVS surfaces telemetry metrics (e.g., *"Notice: 15 learners flagged difficulties understanding fractional distillation in the last 24 hours"*), enabling targeted physical classroom interventions.

## 👥 2.3 The Parent & Public Chamber (Instant Information Officer)

- **Objective:** Eradicate the operational necessity for guardians to travel massive physical distances to the campus grounds to acquire simple administrative clarity.
- **Knowledge Base Injection:** The database layer contains pre-loaded configurations detailing registration schedules, required submission documentation, uniform protocols, calendar milestones, and fee definitions.
- **Execution Framework:** Delivers clean, immediate answers formatted into short text packets to minimize cellular bandwidth requirements, ensuring accessibility for all community members.

## 3. DETAILED TECHNICAL STACK ARCHITECTURE

To ensure production durability, instantaneous rendering states, and industrial-grade key isolation, the system completely bypasses legacy monolithic structures in favor of an enterprise-tier decoupled architecture.

FIGURE 3.1: ASYNCHRONOUS DATA PIPELINE & SECURITY GATEWAYS



### 3.1 Frontend Component Architecture

**React.js powered by Vite Developer Tooling:** Traditional HTML/CSS architectures require full page reloads or intensive DOM mutations that delay the chat viewport interface. React uses an optimized Virtual DOM compilation approach to render arriving stream chunks dynamically, resulting in an experience equivalent to a native mobile interface.

**Tailwind CSS Framework Layout Engine:** Standard CSS files introduce maintenance debt. Tailwind handles deep UI styling at the markup layer using predefined variables: deep dark-blue slate backgrounds (#0b1224) for optimal readability, premium institutional blue boxes (#2563eb) for SKAVS outputs, and clean off-white modules for incoming student inputs.

**Lucide React Component Arrays:** Replaces inaccurate emojis with scalable, lightweight vector SVG graphic elements that load seamlessly on low-tier screens without adding layout weight.

## 3.2 Backend API Processing Pipeline

**Node.js with Express Framework API:** Embedding AI keys or prompt structures inside client-side JavaScript presents an unacceptable security vulnerability, allowing users to extract credentials via standard browser inspector panels. Node.js implements a permanent server-side firewall to shield operational infrastructure.

**Operational Express Middleware:** Cross-Origin Resource Sharing (cors) checks and authorizes explicit client-side requests, while dotenv loads sensitive environment flags directly from private server memory structures.

**Google Generative AI Core Engine Interface:** Connects directly through the native SDK to standard enterprise-tier modules (e.g., gemini-1.5-pro). The API intercepts incoming client messages, attaches the strict system directives associated with the verified active session user classification, and transmits the secure payload to the core model.

# 4. DEPENDENCY MANIFESTS & INFRASTRUCTURE AUTOMATION

## 4.1 Backend Engine Configuration (`backend/package.json`)

```
{
  "name": "skavs-backend-engine",
  "version": "1.0.0",
  "type": "module",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  },
  "dependencies": {
    "@google/generative-ai": "^0.11.0",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

## 4.2 Frontend Workspace Manifest (`frontend/package.json`)

```
{
  "name": "skavs-frontend-interface",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build"
  },
  "dependencies": {
    "axios": "^1.7.2",
    "lucide-react": "^0.379.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.0",
    "autoprefixer": "10.4.19",
    "postcss": "^8.4.38",
    "tailwindcss": "^3.4.3",
    "vite": "^5.2.11"
  }
}
```

## 5. PERFORMANCE FORECASTS & DIAGNOSTICS

Because the application uses a decoupled Single Page Framework that handles presentation layer operations directly inside the client browser runtimes, bandwidth usage is minimized. This system provides maximum resilience across network arrays with limited baseline coverage profiles.

| Performance Telemetry Marker   | Target Engineering Limit | Legacy Portal Baseline          |
|--------------------------------|--------------------------|---------------------------------|
| First Contentful Paint (FCP)   | < 450 Milliseconds       | 2.4 Seconds                     |
| API Endpoint Roundtrip Latency | < 1.2 Seconds            | Manual processing (24-48 Hours) |
| Data Overhead Per Chat Session | < 12 Kilobytes           | > 4.5 Megabytes                 |

## 6. TERMINAL AUTOMATION COMMAND

To instantly instantiate this verified directory skeleton and create all source file allocations cleanly within your local VS Code workspace, run this native execution string inside your terminal environment:

```
mkdir -p backend/src/config backend/src/routes frontend/src/components frontend/src/assets && touch
```